Closer through commonality: Enhancing hypergraph contrastive learning with shared groups

Daeyoung Roh[1], Donghee Han[1], Daehee Kim[1], Keejun Han[2*], Mun Yi[3*]
Graduate School of Data Science, KAIST, Republic of Korea[1]
School of Computer Engineering, Hansung University, Republic of Korea[2]
Department of Industrial and Systems Engineering, KAIST, Republic of Korea[3]
{dybroh, handonghee, dh.kim}@kaist.ac.kr, keejun.han@hansung.ac.kr, munyi@kaist.ac.kr

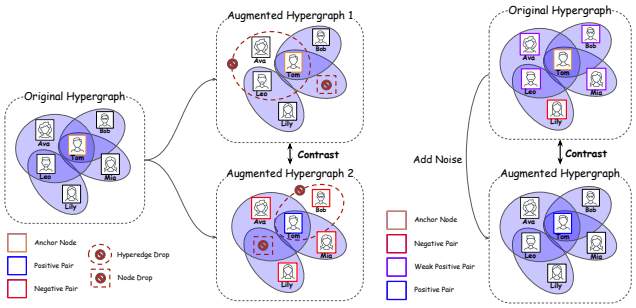
***Abstract*—Hypergraphs provide a superior modeling framework for representing complex multidimensional relationships in the context of real-world interactions that often occur in groups, overcoming the limitations of traditional homogeneous graphs. However, there have been few studies on hypergraph-based contrastive learning, and existing graph-based contrastive learning methods have not been able to fully exploit the high-order correlation information in hypergraphs. Here, we propose a Hypergraph Fine-grained contrastive learning (HyFi) method designed to exploit the complex high-dimensional information inherent in hypergraphs. While avoiding traditional graph augmentation methods that corrupt the hypergraph topology, the proposed method provides a simple and efficient learning augmentation function by adding noise to node features. Furthermore, we expands beyond the traditional dichotomous relationship between positive and negative samples in contrastive learning by introducing a new relationship of weak positives. It demonstrates the importance of fine-graining positive samples in contrastive learning. Therefore, HyFi is able to produce high-quality embeddings, and outperforms both supervised and unsupervised baselines in average rank on node classification across 10 datasets. Our approach effectively exploits high-dimensional hypergraph information, shows significant improvement over existing graph-based contrastive learning methods, and is efficient in terms of training speed and GPU memory cost. The source code is available at https://github.com/Noverse0/HyFi.git.**

***Index Terms*—Hypergraph, Contrastive Learning, Graph Neural Network, Graph Algorithms**

I. Introduction

Hypergraphs surpass simple pairwise connections, capturing complex interactions in the real world [10], [13], [18], [21], [32], [34], [36], [37], [45]. Although recent advances in graph-based learning have primarily focused on these simple relationships, they often fail to account for the intricate group dynamics inherent in real-world data. In contrast, hypergraphs can go beyond pairwise connections to capture high-order data correlations. Their application across diverse fields such as social network analysis [25], [26], [35], bioinformatics [15], [22], [39], [41], and complex system modeling is increasingly prevalent, as they can effectively represent the depth of relationships within these domains. These applications showcase the advantages of hypergraphs in capturing high-order data correlations from real-world scenarios [8], [14], [16], [29], [31].

* Corresponding author

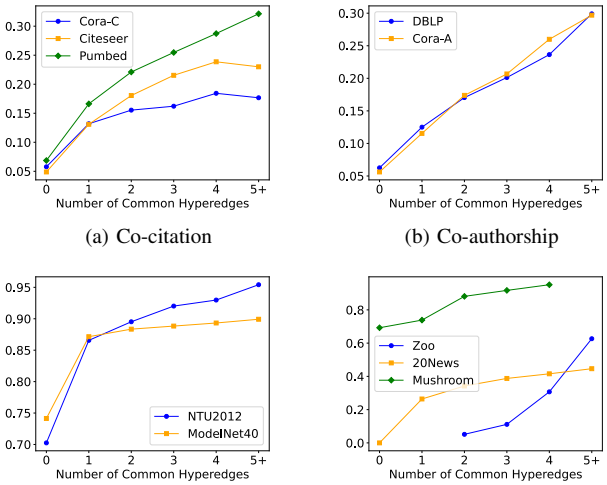


(a) Tranditional HGCL (b) HyFi

Fig. 1: Example of tranditional hypergraph contrastive learning vs. HyFi. In (a), contrastive learning only learns postive or negative pairs between graphs augmented by randomly dropping hyperedges or nodes. In (b), HyFi preserves the topology of the hypergraph by adding noise to the node features, while adding weak postive pair relationships for fine-grained contrastive learning.

Despite its potential, research on hypergraph-based learning has been relatively sparse, with even fewer studies exploring contrastive learning, although the possibility of using a hypergraph has recently gained momentum. The complex structure of hypergraphs, which includes nodes and hyperedges, differs significantly from traditional homogeneous graphs, requiring new approaches tailored specifically to hypergraphs for effective learning. Traditional hypergraph contrastive learning (HGCL) relies on contrastive learning between graphs augmented with random variations in nodes or hyperedges. Consequently, it depends heavily on the embedding of the augmented opposite view. In contrast, we propose **Hy**pergraph **Fi**ne-Grained contrastive learning (HyFi), a novel method that identifies and contrastively learns new positive pairs within the original view without perturbing the topology of the original graph. Furthermore, our model does not rely on the quality of the augmented view, as it generates positive pairs by simply adding some noise to the node features. Fig. 1 illustrates the difference between trainditional HGCL and HyFi.

Traditional graph contrast learning (GCL) typically generates two augmented views of an existing graph and then contrasts them using positive and negative pairs [9], [19],



(a) Co-citation (b) Co-authorship

(c) Computer Vision and Graphics (d) Machine Learning Repository

Fig. 2: The average of the cosine similarity of the node features over the number of commonalities between two nodes, where commonality, defined as a common group, means common hyperedges. The higher the number of common hyperedges, the higher the similarity of the two node features.

[28], [38], [43]. However, these traditional approaches do not adequately capture the nature of hypergraphs, especially their ability to represent complex group interactions and higher-order correlations represented by commonality. The dichotomous relationship represented by the traditional positive pair and negative pair makes sense for homogeneous graphs with one relation, but cannot capture the higher-order correlations present in hypergraphs with many different relations. Furthermore, efficient contrastive learning is sensitive to the augmentation of the graph, which mostly relies on the randomness of the graph, which can lead to loss of important information in the graph.

Therefore, we propose a new relation, called a weak positive pair, which shows how group sharing can be exploited in contrastive learning. We show that a weak positive pair is a relationship between nodes that share the same group relative to an anchor node, and that we see significant performance gains just by introducing weak positive pairs. Moreover, weak positive pairs can be found within the original view, so there is little dependence on augmenting the graph. These new relationships are based on the idea that we can define ourselves by the people around us who are in the same group as us. In reality, we are strongly influenced by the people around us and relate to similar people.

Furthermore, our approach to augmenting the graph by adding noise to the node features to generate positive samples differs from typical graph augmentation techniques that corrupt the original topology and increase model complexity through random modifications. However, we show that this approach is still sufficient to generate positive samples and is an efficient graph augmentation technique. Therefore, we propose a new form of HGCL method that emphasizes the importance of shared hyperedges and shared nodes while maintaining the structural integrity of the hypergraph. Properly exploiting this commonality is very important in HGCL because hypergraphs are well suited to represent these different relationships through hyperedges.

Distinguishing these types of relationships allows us to capture complex intricacies more accurately. This approach is based on the assumption that the more two nodes share a common group, the more similar they are. In other words, we define higher-order correlations in a hypergraph as "commonality," which is the property of sharing a common group between two nodes. Our assumption is confirmed in Figure 2, where we see that the cosine similarity of two node features increases as the number of hyperedges shared by the two nodes increases. For example, two people belonging to the same group could mean that they have the same disposition, or they could belong to the same group and have similar dispositions. Therefore, the assumption of common group membership is important to exploit the high-dimensional correlations in the hypergraph. Also, the model implements group unit contrastive learning to efficiently learn homophily trends within the hypergraph. In previous HGCL, contrastive learning is based on nodes; however, we use group-based contrastive learning. Thus, nodes that belong to the same group are able to contrasted multiple times.

Our proposed HyFi method enhances the classification of weak positive relations in shared group connections between nodes in a hypergraph. This approach generates higher quality embeddings than those produced by traditional hypergraph contrastive learning (HGCL) models. As a result, HyFi outperformed 10 supervised and 6 unsupervised baselines in node classification tasks across 10 datasets. It also outperformed both types of baselines in 10 separate node classification tasks. Furthermore, because HyFi does not modify the topology of the graph, it is widely applicable to a variety of datasets and is more efficient in terms of both time and memory usage compared to traditional HGCL models.

The main contributions of this paper are highlighted as follows:

- We propose a novel contrastive learning method, called HyFi. The method exploits the higher-order correlations in hypergraphs represented by commonality to achieve efficient contrast learning that is less dependent on graph augmentation than existing methods.
- We introduce a new relation called weak positive pair, which shows that setting the positive sample to be fine-grained is key to contrastive learning.
- We propose an augmentation scheme that adds noise to the node features in a way that does not preserve the topology of the hypergraph, and show that it is sufficient to generate positive samples that are comparable to various forms of augmentation.

II. Related Work

A. Hypergraph neural networks

Hypergraph neural networks (HGNNs) extend graph neural networks(GNNs) by enabling complex interactions through hyperedges. Early models like HGNN [7] and HyperGCN [33] convert hypergraphs into graphs for processing, leading to the loss of higher-order relationships. HNHN [5] improves this problem by using a two-step message passing that preserves these relationships. Attention-based models such as HCHA [1] further refine node and hyperedge aggregations. Advanced frameworks such as UniGNN [11] and AllSet [3] have generalized these concepts to hypergraphs, with UniGCNII focusing on preventing over-smoothing in deep networks. AllSet and its variant AllSetTransformer use sophisticated functions for improved aggregation. However, most models still struggle with fully capturing the nuanced interactions within hyperedges. Furthermore, a majority of these models are still primarily limited to the settings of supervised or semi-supervised learning.

B. Graph contrastive learning

The realm of graph contrastive learning has been largely inspired by its success in computer vision and natural language processing. Techniques like DGI [28], InfoGraph [24], GRACE [43], and MVGRL [9] have shown promising results in obtaining expressive representations for graphs and nodes by maximizing mutual information between various levels of graph structure. More recent approaches like GCA [44] and DGCL [42] have introduced novel methods for data augmentation, which is crucial for contrastive learning. However, these methods often require manual trial-and-error, cumbersome search processes, or expensive domain knowledge for augmentation selection, limiting their scalability and general applicability.

C. Hypergraph contrastive learning

Contrastive learning on hypergraphs, though still in its nascent stages, is beginning to receive more attention. Initial studies such as S^2-HHGR [40] have focused on applying contrastive learning for specific applications like group recommendation, employing unique hypergraph augmentation schemes. However, they often overlook the broader potential of hypergraph structures in general representation learning. Recent efforts in this area are starting to explore more comprehensive approaches that fully leverage the complex structure of hypergraphs. For instance, TriCL [12] uses a tri-directional contrast mechanism to learn node embeddings by contrasting node, group, and membership labels. However, many of these methods still focus on specific applications and do not fully exploit the rich, multi-relational nature of hypergraphs.

In this paper, we propose a hypergraph-specific contrastive learning method that exploits the high-order correlations of hypergraphs by presenting a fine-grained approach to learning node pairs. We also present a method that does not exploit the structure of hypergraphs, enabling more efficient learning than existing hypergraph models.

TABLE I: Primary Notations in this paper.

Notation	Definition
$G=(V,E)$	Hypergraph with nodes V and hyperedges E
H	Incidence matrix
X	Node feature matrix
V	Hypergraph with nodes, where $V=\{v_1,v_2,\ldots,v_{\lvert V\rvert}\}$
E	Hypergraph with hyperedges, where $E=\{e_1,e_2,\ldots,v_{\lvert E\rvert}\}$
$P^{(l)}$	Node embedding matrix at layer l
$Q^{(l)}$	Hyperedge embedding matrix at layer l
l	Number of layers
D_v	Diagonal matrix, Degree of node
D_e	Diagonal matrix, Degree of hyperedge
W	Diagonal matrix, Weight of hyperedge
$\mathcal{N}(0,\sigma^2)$	noises from Gaussian distribution with 0 mean and variance σ^2
$f_\theta(\cdot)$	HGNN Encoder
$g_\pi(\cdot)$	Node embedding projection head
$g_\psi(\cdot)$	Hyperedge embedding projection head
Z	Final Node embedding after passing the projection head
Y	Final Hyperedge embedding after passing the projection head
$sim(u,v)$	Cosine similarity $\mathbf{u}^\mathsf{T}\mathbf{v}/\lVert\mathbf{u}\rVert\lVert\mathbf{v}\rVert$
N_{v_i}	Set of other nodes in the group to which node v_i belongs
M	Number of Noise views

III. Preliminary

A. Preliminaries and Notation

In this section, we introduce essential concepts relevant to HyFi, and present preliminaries concerning hypergraphs and hypergraph neural networks. The notations utilized are concisely summarized in Table I.

1) Hypergraphs: HyFi employs hypergraphs as a foundational structure, denoted by $G=(V,E)$, where G represents the hypergraph, V the set of nodes, and E the set of hyperedges. The nodes and hyperedges are denoted as $V=\{v_1,v_2,\ldots,v_{|V|}\}$ and $E=\{e_1,e_2,\ldots,e_{|E|}\}$, respectively. The node feature matrix is defined by $X\in\mathbb{R}^{|V|\times d}$, where each node v_i's feature vector x_i is represented by $X[i,:]^T\in\mathbb{R}^d$. In general, a hypergraph can be alternatively characterized by its incidence matrix $H\in\{0,1\}^{|V|\times|E|}$, where the entries are defined as $h_{ij}=1$ if node v_i is included in hyperedge e_j, and $h_{ij}=0$ otherwise.

2) Hypergraph neural networks: Prior studies on HGNNs have predominantly adopted a two-phase neighborhood aggregation approach that includes both node-to-hyperedge and hyperedge-to-node aggregations. These networks use an iterative process to update hyperedge representations by accumulating the representations of neighboring nodes, and similarly, to update node representations by accumulating those of neighboring hyperedges. At the k-th layer, the representations of nodes and hyperedges are denoted as $P^{(k)}\in\mathbb{R}^{|V|\times d'_k}$ and $Q^{(k)}\in\mathbb{R}^{|E|\times d''_k}$, respectively. More precisely, the k-th layer function in a Hypergraph neural network can be expressed as $q_j^{(k)}=f^{(k)}_{V\to E}(q_j^{(k-1)},p_i^{(k-1)}:v_i\in e_j)$ and $p_i^{(k)}=f^{(k)}_{E\to V}(p_i^{(k-1)},q_j^{(k)}:v_i\in e_j)$, where $p_i^{(0)}=x_i$. Therefore, the choice of aggregation functions $f_{V\to E}(\cdot)$ and $f_{E\to V}(\cdot)$ is critical to the performance of the model. We will use HGNN model [7] as the backbone of the HyFi encoder.

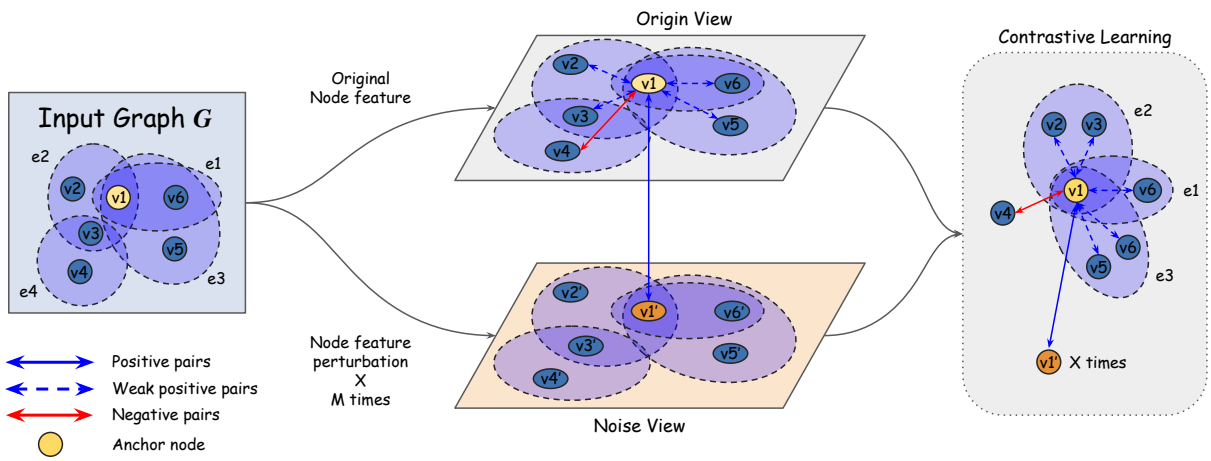


Fig. 3: Illustration of the node-level contrastive loss calculation process in HyFi. HyFi uses both the origin view and the noise view to calculate contrastive loss. In the origin view, there is a negative and a weak positive pair, and in the noise view there is a positive pair, which is a node in the same position as the anchor node.

IV. Proposed Method

In this section, we provide a detailed exposition of our proposed HyFi, the overall framework of which is illustrated in Fig. 3.

A. HyFi

Here, we propose HyFi, a method that exploits the high-dimensional information of hypergraphs for fine-grained contrastive learning by introducing weak positive pairs that share the same group. Unlike existing methods in hypergraph contrastive learning, HyFi fully exploits the homophily feature of hypergraphs, which is based on group commonality, to achieve superior performance. In addition, it avoids augmentations that modify the topology of the hypergraph. Instead, HyFi adds Gaussian random noise to the node features, allowing for fast training speeds, efficient memory usage, and robustness against constraints typically imposed by graph augmentation.

B. HyFi: Node perturbation

When we have a hypergraph $G = (V, E)$ along with node features X, our approach involves perturbing only X without altering the topology of the hypergraph. The perturbation is executed by adding Gaussian-distributed random noise to the original node features. Since the information values of the Node feature are mostly one-hot vectors or normalized between 0 and 1, we perturb the values so that they are between 0 and 1. Our proposed method to perturb the node feature X can be described as,

$$X' = X + (-1)^X \cdot |\epsilon|, \quad \epsilon \sim \mathcal{N}(0, \sigma^2) \tag{1}$$

, where X and X' are the original node feature and the node feature with noise added, respectively. ϵ denotes noise with a Gaussian distribution with a mean of zero and a variance of σ^2.

C. HyFi: Hypergraph encoder

In our framework, we employ an HGNN model for the hypergraph encoder, denoted by $f_\theta(\cdot)$. As input to the encoder, we take the same hypergraph $G = (V, E)$, but different original node features X and node features with noise added X'. We denote the view generated with X as the origin view, and the view generated with X as the noise view. The encoder generates embeddings for the nodes and hyperedges of the hypergraph, respectively. The representation of this process is as follows:

$$\begin{aligned} f_\theta(G, X) &= (P, Q), \\ f_\theta(G, X') &= (P', Q'), \end{aligned} \tag{2}$$

, where P and Q represent the embeddings of nodes and hyperedges from the original view, respectively, while P' and Q' correspond to the embeddings from the noise view.

$$\begin{aligned} Q^{(k)} &= \sigma\left(D_E^{-1} H^T D_V^{-1/2} P^{(k-1)} \Theta_E^{(k)} + b_E^{(k)}\right), \\ P^{(k)} &= \sigma\left(D_V^{-1/2} H W Q^{(k)} \Theta_V^{(k)} + b_V^{(k)}\right), \end{aligned} \tag{3}$$

, where $P^{(0)} = X$ and W is the identity matrix.

The encoder generates embeddings for both the nodes and hyperedges of the hypergraph, and we incorporate two projection heads, symbolized by $g_\phi(\cdot)$ and $g_\psi(\cdot)$, to map the embeddings of nodes and hyperedges, respectively. These projection heads are built using a two-layer MLP with ELU activation function [4] as,

$$\begin{aligned} Z &:= g_\phi(P) \quad Z' := g_\phi(P'), \\ Y &:= g_\psi(Q) \quad Y' := g_\psi(Q'), \end{aligned} \tag{4}$$

, where $\prime$ is the embedding of the noise view from the perturbed node features.

D. HyFi: Contrastive Loss

HyFi computes contrastive loss at both the node and edge levels. The core principle of this computation is that nodes sharing hyperedges are considered weak positive pairs in node-level contrastive learning. This assumption is based on the premise that nodes within the same group are likely to share similarities. Furthermore, we exploit the unique perspective of each group by performing the contrastive loss computation specifically within each group. This method allows us to focus on a node's group affiliation, allowing us to compute the contrastive loss proportional to the number of hyperedges shared between two nodes. We call this method group-unit contrastive learning. Computing contrastive loss at the edge level uses information from the nodes that belong to each hyperedge; therefore, the computational process is the same at the node level and at the edge level.

1) Node-level contrastive loss: We consider a view with the original node features, X, as the original view and the noisy node features, X', as the noisy view. When calculating contrastive loss at the node level, we consider three types of pairs: positive pairs, weakly positive pairs, and negative pairs. The number of hyperedges shared between each node can be determined as the matrix product of the incidence matrix H and its transpose H^T. Here, the diagonal elements of the resulting matrix represent the number of hyperedges each node belongs to, while the off-diagonal elements represent the number of hyperedges shared between nodes i and j. Mathematically, this can be expressed as:

$$E = H \cdot H^T$$

where E is the resulting matrix. In E, the element E_{ii} represents the number of hyperedges node i belongs to, and the element E_{ij} (for $i \neq j$) represents the number of hyperedges shared between nodes i and j.

Nodes that share the same hyperedge in the original view are considered weak positive pairs. All other nodes, which do not share a hyperedge with the anchor node, are categorized as negative pairs. For example, if a weak positive pair node represents a relationship within the same group as me, then a node that belongs to a group I have never interacted with is defined as a negative pair. Furthermore, we utilize the noise view embedding—obtained by perturbing the node features earlier—as a positive pair. Consequently, the number of positive pairs equals the number of noise views.

We also assign a weight to the weak positive pair, which represents the probability that a node is a positive pair with the anchor node. This weight is calculated as the ratio of the number of shared hyperedges between the two nodes to the total number of hyperedges of the anchor node. For instance, if the anchor node is part of 3 hyperedges and shares 2 hyperedges with another node, the weight would be 2/3. The contrast loss is also calculated based on the number of shared hyperedges, allowing us to compute the loss on a group basis. Therefore, the final weight for the last weak positive pair is determined by multiplying the number of shared hyperedges by the ratio of the number of shared hyperedges identified earlier. The formula for the weights of weak positive is as follows:

$$w_{i,j}^{pos} = \underbrace{|E_{i,j}|}_{\text{for group unit contrast}} \cdot \underbrace{\frac{|E_{i,j}|}{|E_{i,i}|}}_{\text{postivie probability}}, \tag{5}$$

Here, $w_{i,j}^{pos}$ denotes the weak positive weight. The contrastive losses for $\mathcal{L}_n^{pos}$, $\mathcal{L}_n^{weak_pos}$, and $\mathcal{L}_n^{neg}$ are calculated as follows:

$$\begin{aligned} \mathcal{L}_n^{pos} &= \sum_{m \in M} e^{\text{sim}(z_i, z'_{m,i})/\tau_n}, \\ \mathcal{L}_n^{weak_pos} &= \sum_{j \in N_{z_i}} w_{i,j}^{pos} \cdot e^{\text{sim}(z_i, z_j)}/\tau_n, \\ \mathcal{L}_n^{neg} &= \sum_{j \notin N_{z_i}} e^{\text{sim}(z_i, z_j)}/\tau_n, \end{aligned} \tag{6}$$

, where τ_n is a temperature parameter, M is set of noise views.

With the Origin View at the anchor z_i and the set of Noise View as z'_m. The node-level contrastive loss $\mathcal{L}_n(z_i, z'_m)$ is defined as follows:

$$\mathcal{L}_n(z_i, z'_m) = -\log\left(\frac{\mathcal{L}_n^{pos} + \mathcal{L}_n^{weak_pos}}{\mathcal{L}_n^{pos} + \mathcal{L}_n^{weak_pos} + \mathcal{L}_n^{neg}}\right) \tag{7}$$

.

2) Edge-level contrastive loss: The calculation of edge-level contrastive loss is the same process as the calculation of node-level contrastive loss. However, at the edge level, the node and hyperedge relationships at the node level are reversed, i.e., at the hyperedge level, the shared nodes between hyperedges are utilized instead of the shared hyperedges between each node.

3) Final contrastive loss: Finally, the final loss $\mathcal{L}$ is calculated by summing the node-Level contrastive loss $loss_n$ and the edge-level contrastive loss $loss_e$ as follows:

$$\mathcal{L} = \sum_{i=1}^{V} \mathcal{L}_n(z_i, z'_m) + \alpha \cdot \sum_{i=1}^{E} \mathcal{L}_e(y_i, y'_m) \tag{8}$$

, where α are the weights for the edge-level loss.

E. Complexity Analysis

The training algorithm for HyFi is thoroughly outlined in Algorithm 1. Additionally, we conducted an analysis of the time and space complexity of the algorithm.

1) Time Complexity: We assess time complexity based on the pseudocode. Initially, the generation of noise features in line 8 has a time complexity of $O(m \cdot n_d \cdot |V|)$, where n_d is the dimensionality of the node features, $|V|$ represents the number of nodes, and m is the number of noise views. For the computation of the node-level contrastive loss in line 12, the cosine similarity for each node is calculated. This includes computing the similarity within the original view and between

Algorithm 1 HyFi

Input: Hypergraph $G = (V, E)$, Node Feature X
Parameters: weight for edge loss α, weight matrix for node W_v, weight matrix for edge W_e, standard deviation of Gaussian noise σ^2, number of noise views m
Output: Trained HGNN encoder f_Θ, node embeddings Q, edge embeddings P
for $epoch \leftarrow 1, 2, \ldots$ **do**
  $Q, P \leftarrow f_\Theta(G, X)$
  $Q_{proj}, P_{proj} \leftarrow g_\Phi(Q), g_\Psi(P)$
  for $i \leftarrow 1$ to m **do**
    $X' \leftarrow X + \mathcal{N}(0, \sigma^2)$
    $Q', P' \leftarrow f_\Theta(G, X')$
    $Q'_{proj}, P'_{proj} \leftarrow g_\Phi(Q'), g_\Psi(P')$
  end for
  $\mathcal{L}_{node} \leftarrow Loss(Q_{proj}, Q'_{proj}, W_v)$
  $\mathcal{L}_{edge} \leftarrow Loss(P_{proj}, P'_{proj}, W_e)$
  $\mathcal{L} \leftarrow \mathcal{L}_{node} + \alpha \cdot \mathcal{L}_{edge}$
  Perform gradient descent on Θ to minimize $\mathcal{L}$
end for

the original view and the noise view. Cosine similarity calculation has a complexity of $O(n_d)$ per node. When computed for all nodes and for each noise view, the total time complexity of node-level contrastive loss is $O(m \cdot |V| \cdot n_d)$. Similarly, the computation of the edge-level contrastive loss follows the node-level process but applies to the edge embeddings P, resulting in a time complexity of $O(m \cdot |E| \cdot n_d)$, where $|E|$ denotes the total number of hyperedges. Therefore, the time complexity per training iteration is $O(m \cdot n_d \cdot (|V| + |E|))$. All calculations for HyFi can be efficiently performed on a GPU.

2) Space Complexity: The graph consists of a set of nodes V and a set of edges E. Depending on the representation of the graph, which could be an adjacency list or matrix, the space required is typically $O(|V| + |E|)$. Node features occupy a space complexity of $O(|V| \cdot n_d)$ for a n_d-dimensional feature vector per node. Node and edge embeddings, which correspond to vectors for each node and edge, occupy $O(|V| \cdot n_d)$ and $O(|E| \cdot n_d)$, respectively.

The weight matrices for nodes and edges, necessary for calculating the weights for weak positive and weak negative pairs, require a space of $O(|V| \cdot n_d)$ and $O(|E| \cdot n_d)$, respectively. The new feature sets generated by adding noise to each node feature match the size of the original features, contributing an additional $O(m \cdot |V| \cdot n_d)$ to the space complexity, where n is the number of noise views. Thus, the total space complexity of the algorithm is the sum of these components.

When considering the most significant terms in the space complexity, the weight matrices for nodes and edges are $O(|V| \cdot n_d)$ and $O(|E| \cdot n_d)$, respectively. HyFi maintains the original graph topology, only augmenting the node feature X according to the noise view, facilitating more efficient memory usage compared to models that augment the graph topology itself.

TABLE II: Statistics for datasets.

Dataset	# Nodes	# Hyperedges	# Classes
Cora-C	1,434	1,579	7
Citeseer	1,458	1,079	6
Pubmed	3,840	7,963	3
Cora-A	2,388	1,072	7
DBLP	41,302	22,363	6
Zoo	101	43	7
20News	16,242	100	4
Mushroom	8,124	298	2
NTU2012	2,012	2,012	67
ModelNet40	12,311	12,311	40

V. Experiments

In this section, we conduct experiments to evaluate HyFi through answering the following research questions.

- **RQ1 (Generalizability)**: How does HyFi outperform baselines on node classification tasks?
- **RQ2 (Efficiency)**: How efficient is HyFi regarding GPU memory usage and training time?
- **RQ3 (Ablation Study)**: How do the newly introduced modules in HyFi affect its overall performance?
- **RQ4 (Type of Augmentation)**: How do different augmentation methods affect performance?
- **RQ5 (Effect of Positive sample)**: How does the number of positive samples affect hypergraph contrastive learning?

A. Experimental Setup

1) Datasets: To compare the performance of HyFi, we use a total of 10 datasets that can be categorized into: (1) citation network datasets (Cora-C, Citeseer, and Pubmed [23]), (2) co-authorship network datasets (Cora-A and DBLP [20]), (3) computer vision and graphics datasets (NTU2012 [2] and ModelNet40 [30]), and (4) datasets from the UCI Categorical Machine Learning Repository (Zoo, 20Newsgroups, and Mushroom [6]). The statistics of these datasets are provided in Table II.

2) Hyperparameter Setting: For all datasets, we used AdamW as the optimizer [17]. To perturb the node features, we use random noise from a Gaussian distribution with mean zero and standard deviation σ^2 as a hyperparameter. The number of positive samples equals the number of noise views, which is also a hyperparameter.

3) Compared baselines: HyFi was compared against 10 (semi-)supervised models and 5 unsupervised models. Given that graph-based methods are not directly applicable to hypergraphs, hypergraphs were transformed into graphs through clique expansion before applying these methods. We focused particularly on comparisons with the state-of-the-art model in HGCL, which is TriCL. To ensure a fair comparison, we assume the use of the same encoder (HGNN) architecture. For the baselines other than TriCL, results from their respective references are cited.

4) Evaluation protocol: For the node classification task, we follow the standard linear evaluation protocol introduced in [28]. First, an encoder is trained in a fully unsupervised

TABLE III: Comparison of node classification accuracy. The highest scores in each dataset are highlighted in bold, indicating the best performance by method, while the second highest scores are underlined. The 'A.R.' indicates the average rank across all datasets. '-' indicates that the results are not available in published papers.

	Method	Cora-C	Citeseer	Pubmed	Cora-A	DBLP	Zoo	20News	Mushroom	NTU2012	ModelNet40	A.R.↓
Supervised	MLP	60.32±1.5	62.06±2.3	76.27±1.1	64.05±1.4	81.18±0.2	75.62±9.5	79.19±0.5	99.58±0.3	65.17±2.3	93.75±0.6	11.4
	GCN	77.11±1.8	66.07±2.4	82.63±0.6	73.66±1.3	87.58±0.2	36.79±9.6	-	92.47±0.9	71.17±2.4	91.67±0.2	10.7
	GAT	77.75±2.1	67.62±2.5	81.96±0.7	74.52±1.3	88.59±0.1	36.48±10.0	-	-	70.94±2.6	91.43±0.3	10.0
	HGNN	77.50±1.8	66.16±2.3	83.52±0.7	74.38±1.2	88.32±0.3	78.58±11.1	80.15±0.3	98.59±0.5	72.03±2.4	92.23±0.2	7.1
	HyperConv	76.19±2.1	64.12±2.6	83.42±0.6	73.52±1.0	88.83±0.2	62.53±14.5	79.83±0.4	97.56±0.6	72.62±2.6	91.84±0.1	8.7
	HNHN	76.21±1.7	67.28±2.2	80.97±0.9	74.88±1.6	86.71±1.2	78.89±10.2	79.51±0.4	99.78±0.1	71.45±3.2	92.96±0.2	7.9
	HyperGCN	64.11±7.4	59.92±9.6	78.40±9.2	60.65±9.2	76.59±7.6	40.86±2.1	77.31±6.0	48.26±0.3	46.05±3.9	69.23±2.8	14.1
	HyperSAGE	64.98±5.3	52.43±9.4	79.49±8.7	64.59±4.3	79.63±8.6	40.86±2.1	-	-	-	-	13.7
	UniGCN	77.91±1.9	66.40±1.9	**84.08±0.7**	77.30±1.4	90.31±0.2	72.10±12.1	**80.24±0.4**	98.84±0.5	73.27±2.7	94.62±0.2	5.0
	AllSet	76.21±1.7	67.83±1.8	82.85±0.9	76.94±1.3	90.07±0.3	72.72±11.8	79.90±0.4	99.78±0.1	75.09±2.5	96.85±0.2	5.0
Unsupervised	Random-Init	63.62±3.1	60.44±2.5	67.49±2.2	66.27±2.2	76.57±0.6	78.43±11.0	77.14±0.6	97.40±0.6	74.39±2.6	96.29±0.3	10.9
	Node2vec	70.99±1.4	53.85±1.9	78.75±0.9	58.50±2.1	72.09±0.3	17.02±4.1	63.35±1.7	88.16±0.8	67.72±2.1	84.94±0.4	14.6
	DGI	78.17±1.4	68.81±1.8	80.83±0.6	76.94±1.1	88.00±0.2	36.54±9.7	-	-	72.01±2.5	92.18±0.2	8.2
	GRACE	79.11±1.7	68.65±1.7	80.08±0.7	76.59±1.0	-	37.07±9.3	-	-	70.51±2.4	90.68±0.3	9.4
	S^2-HHGR	78.08±1.7	68.21±1.8	82.13±0.6	78.15±1.1	88.69±0.2	**80.06±11.1**	79.75±0.3	97.15±0.5	73.95±2.4	93.26±0.2	5.7
	TriCL	81.03±1.3	71.97±1.3	83.80±0.6	**82.22±1.1**	**90.93±0.2**	79.47±11.0	79.93±0.2	98.93±0.3	74.63±2.5	97.33±0.1	2.5
	HyFi	**81.48±1.5**	**72.57±1.1**	83.82±0.6	79.33±1.2	90.41±0.2	80.02±10.9	79.76±0.3	**99.79±0.2**	**75.10±2.6**	**97.38±0.1**	**1.9**

manner to generate node representations. A simple linear classifier is then trained on top of these fixed representations without backpropagation the gradient through the encoder. For all datasets, nodes are randomly assigned to the training, validation, and test sets in proportions of 10%, 10%, and 80%, respectively, following the approach in [43] and [27]. In the unsupervised setting, we evaluated the model using 20 dataset splits with 5 random weight initializations and reported the average accuracy for each dataset. In the supervised setting, 20 dataset splits were used, each with a different model initialization, and the average accuracy was reported. The experiment was repeated a total of five times, and the average results are reported here.

B. Generalizability (RQ1)

To evaluate the overall performance of HyFi, we performed comparative node classification experiments using different baseline models. As shown in the results in Table III, the HyFi model performed well on the node classification task in both supervised and unsupervised models. HyFi consistently ranked among the top performers on a variety of datasets and had the highest average ranking. Notably, it outperformed TriCL on average ranking, which is encouraging because HyFi outperforms TriCL while still using resources efficiently. These results highlight the importance of considering commonality, which is high-dimensional information in hypergraphs, and suggest that weak positive pair relationships are efficient for contrast learning. HyFi also has the advantage of being applicable to a wide variety of graphs because it relies little on graph augmentation. In fact, it ranks in the top 2 on almost all of the 9 datasets except 20News, and even on 20News, the performance gap to the top is very small.

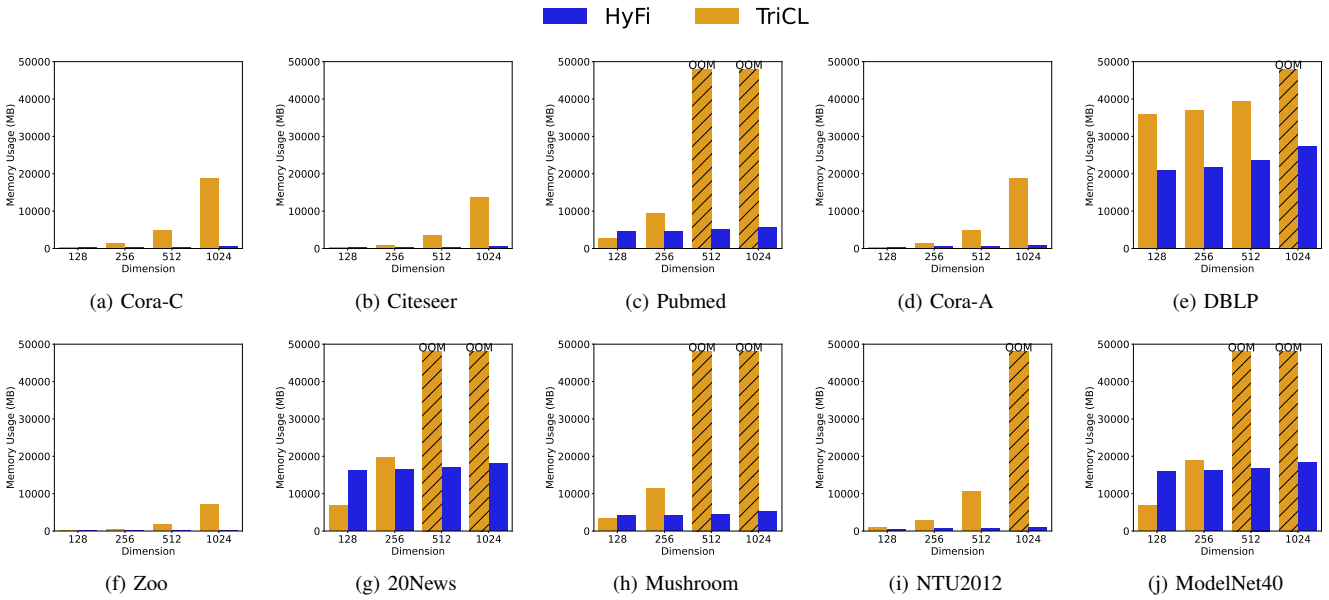
TABLE IV: Comparisons of training time.

Dataset	TriCL	HyFi
Cora-C	21.79 s (×1.71)	12.71 s (×1.00)
Citeseer	58.91 s (×3.17)	18.60 s (×1.00)
Pubmed	350.68 s (×5.07)	69.22 s (×1.00)
Cora-A	33.82 s (×1.40)	24.21 s (×1.00)
DBLP	1145.62 s (×2.38)	481.86 s (×1.00)
Zoo	5.11 s (×2.61)	1.96 s (×1.00)
20News	286.86 s (×5.68)	50.47 s (×1.00)
Mushroom	251.69 s (×7.95)	31.67 s (×1.00)
NTU2012	44.08 s (×4.98)	8.86 s (×1.00)
ModelNet	213.52 s (×3.29)	65.04 s (×1.00)

C. Efficiency (RQ2)

To demonstrate the efficiency of the HyFi model, we conducted an experiment to compare it with another HGCL model, TriCL, in terms of training time and memory cost. To ensure a fair and consistent comparison, we set the hyperparameters between the two models as similar as possible. We experimented on the RTX A6000 (48GB GPU), and only employed batch processing in for the DBLP datasets, as not using batch processing would result in out-of-memory error.

1) Training time: Table IV shows the results of training speed comparisons between the HyFi model and the TriCL model across 10 experimental datasets, highlighting HyFi's significant efficiency advantage. HyFi consistently exhibits faster training times compared to TriCL on these datasets. The disparity in training speeds is particularly pronounced in larger datasets such as PubMed and DBLP. HyFi completes training in just 69.22 seconds and 481.86 seconds for these datasets, respectively, significantly faster than TriCL, which takes 350.68 seconds and 1145.62 seconds. Such results underscore the efficiency of HyFi, especially in scenarios where rapid model



(a) Cora-C (b) Citeseer (c) Pubmed (d) Cora-A (e) DBLP

(f) Zoo (g) 20News (h) Mushroom (i) NTU2012 (j) ModelNet40

Fig. 4: Illustration of the GPU memory usage comparison between HyFi and TriCL, focusing on how they differ in memory consumption in various dimensions, where dimension denotes the encoder output dimension and the project head output dimension, a bar with a diagonal pattern means 'Out of Memory'.

TABLE V: Ablation study exploring various configurations of the contrastive learning framework 'w/o' stands for 'without'.

	Cora-C	Citeseer	Pubmed	Cora-A	DBLP	Zoo	20News	Mushroom	NTU2012	ModelNet40	A.R.↓
HyFi	**81.48±1.5**	72.57±1.1	**83.82±0.6**	**79.33±1.2**	90.41±0.2	**80.02±10.9**	**79.76±0.3**	99.79±0.2	**75.10±2.6**	**97.38±0.1**	**1.4**
w/o weak positive pair	70.89±3.6	60.12±3.6	79.91±0.9	75.75±1.1	88.46±0.1	79.79±10.9	79.69±0.3	**99.87±0.1**	74.88±2.6	97.31±0.1	4.2
w/o positive pair	77.96±1.7	71.97±1.1	83.59±0.6	76.65±1.4	87.62±0.6	79.94±11.0	79.74±0.3	99.79±0.1	74.91±2.6	97.35±0.1	3.2
w/o weak weight	80.44±1.3	**72.61±1.1**	83.79±0.7	78.44±1.1	**90.54±0.1**	79.77±10.9	**79.76±0.3**	99.76±0.2	74.52±2.5	97.32±0.1	2.9
w/o edge loss	81.25±1.4	72.42±1.2	83.77±0.7	79.04±1.1	90.31±0.2	79.79±11.0	79.74±0.3	99.79±0.2	74.94±2.6	97.26±0.1	2.8

training is essential. The consistent speed advantage across various datasets highlights the optimized nature of the HyFi model.

2) Memory cost: As shown in Figure 4, the HyFi model shows significant improvements in GPU memory efficiency compared to the TriCL model across the data sets, without exception. Unlike TriCL, which shows a steep increase in memory consumption at larger dimensions, HyFi maintains a relatively moderate growth in GPU memory consumption. This difference becomes more pronounced at larger dimensions, where TriCL experiences a sensitive increase in memory consumption, leading to out-of-memory problems on datasets such as DBLP, Mushroom, NTU2012 and ModelNet40. HyFi, on the other hand, never runs out of memory on any dataset.

The high efficiency of HyFi is due to two main factors: the strategy of avoiding augmentation in the hypergraph topology and the simplification of the contrastive learning process from three losses in TriCL to only two in HyFi (no need for membership-level loss). As a result, the significant difference in memory consumption between HyFi and TriCL demonstrates the strength of HyFi in handling large datasets or data with high dimensionality of node features.

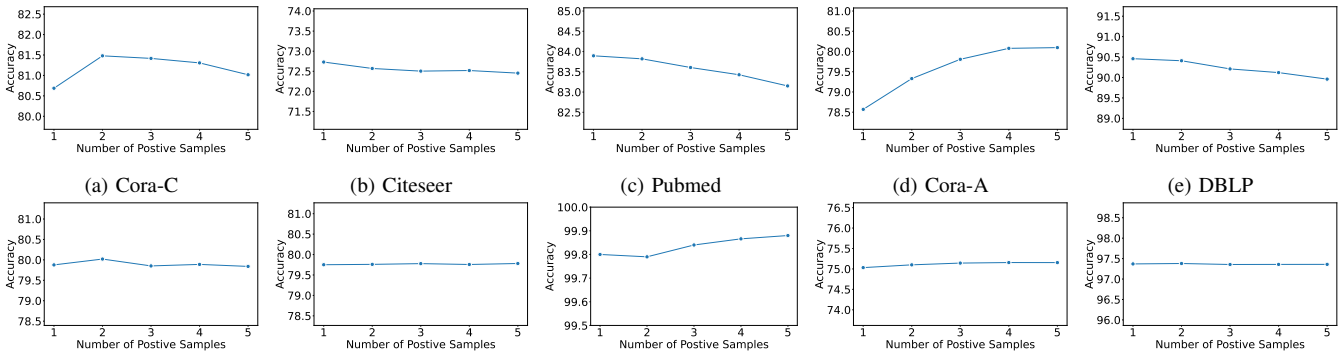
D. Ablation Study (RQ3)

The ablation study, detailed in Table V, evaluates the efficacy of various components on the performance of the HyFi. The study considers four different configurations: a model that does not use weak positive pairs (only positive and negative pairs), a model that does not use positive pairs (only weak positive and negative pairs), a model that does not utilize weights for weak positive pairs, and a model with no edge-level loss.

The results for the model that does not use weak positive pairs show a significant decrease in performance, which is more pronounced than the decrease observed in the model without positive pairs. Except for a few datasets, the performance of the model using only weak positive pairs is almost as good as when using both positive and weak positive pairs. These results demonstrate that the commonality of sharing the same group can be very important in hypergraph contrastive learning. We also observe a slight performance drop when weights for weak positive pairs are not used or when edge-level loss is not utilized. The results indicate that weighted contributions further enhance the model's efficiency. Additionally, we observe that edge-level loss also helps to improve the model's performance.

TABLE VI: Performance of different augmentation method on various datasets.

Augmentation Type	Method	Cora-C	Citeseer	Pubmed	Cora-A	DBLP	Zoo	20News	Mushroom	NTU2012	ModelNet40	A.R.↓
Add Noise	Gaussian Noise	**81.48±1.5**	**72.57±1.1**	83.82±0.6	**79.33±1.2**	**90.41±0.2**	**80.02±10.9**	**79.76±0.3**	99.79±0.2	**75.10±2.6**	**97.38±0.1**	**1.4**
	Uniform Noise	81.40±1.2	72.51±1.1	**84.03±0.7**	77.61±1.2	90.37±0.2	79.65±10.8	**79.76±0.3**	99.79±0.1	74.52±2.5	97.32±0.1	2.9
	Bernoulli Noise	76.78±1.5	71.17±1.3	82.54±0.7	76.78±1.2	90.01±0.2	79.60±10.8	**79.76±0.3**	99.78±0.1	74.40±2.5	97.29±0.1	5.2
Drop	Incidence	78.81±1.5	72.22±1.2	83.59±0.6	77.54±1.3	89.12±0.4	79.91±10.9	79.73±0.3	99.82±0.1	75.05±2.6	97.37±0.1	3.5
	Node	78.89±1.5	72.25±1.2	83.58±0.6	77.57±1.3	89.05±0.4	79.90±10.9	79.73±0.3	99.82±0.1	75.08±2.6	**97.38±0.1**	3.1
	Hyperedge	78.71±1.5	72.25±1.1	83.58±0.6	77.30±1.4	88.86±0.4	79.98±11.0	79.73±0.3	**99.83±0.1**	75.04±2.6	97.37±0.1	3.7



(a) Cora-C (b) Citeseer (c) Pubmed (d) Cora-A (e) DBLP

(f) Zoo (g) 20News (h) Mushroom (i) NTU2012 (j) ModelNet40

Fig. 5: Illustration of the performance of node classification as the number of positive samples. The number of noise views is equal to the number of positive samples in the contrastive learning.

E. Types of Augmentation (RQ4)

We experimented with different augmentation methods to determine the suitability of adding Gaussian noise to node features to generate a positive sample. We experimented with various types of noise (Gaussian, Uniform, Bernoulli). Bernoulli noise perturbs the node feature at values of 0 and 1, which perturbs the node feature significantly compared to Gaussian noise and uniform noise. We also compared our method with common graph augmentation techniques, such as drop augmentation, which involve removing the incidence, node, or hyperedge, thereby modifying the topology of the hypergraph. We used a drop rate of 20%.

Among all graph augmentation methods, adding Gaussian noise achieved the best average ranking. However, since graph augmentation in HyFi is basically aimed at creating positive pairs, adding some noise to the node features is enough to create positive pairs. However, methods that modify the topology of the graph performed poorly in terms of average ranking, suggesting that graph augmentation methods that change the topology are less effective at generating positive samples. Furthermore, the topology of the graph relies heavily on randomness, suggesting the possibility of losing important information. For example, the dropping of a community called family will result in a significant loss of information for some nodes. As such, HyFi's graph augmentation method has the advantage of being able to be used on a variety of graphs without worrying about information loss due to variations in the topology of the graph. Therefore, HyFi, which does not disturb the topology of the graph, can be utilized on a variety of graphs and in a variety of downstream tasks.

F. Effect of Positive sample (RQ5)

Figure 5 shows how performance varies with the number of positive samples. In HyFi, we use 2 positive samples: the number of positive samples is equal to the number of noise views, and the number of positive samples is considered a hyperparameter. We found that the number of positive samples has little effect on the overall performance. We observed less than a 1% difference in performance for most datasets, and it was difficult to find a correlation between the number of positive samples and performance. This indicates that HyFi's performance is not driven by the relationship of weak positive pairs, and that the relationship of positive pairs is not critical to HyFi's performance.

VI. Conclusion

This study has successfully demonstrated the efficacy of the HyFi model in hypergraph-based contrastive learning, demonstrating its ability to capture the nuanced relationships inherent in hypergraph results in higher quality embeddings, and superior performance in node classification task across a variety of datasets. Our novel augmentation technique developed for hypergraphs preserves the original topology while adding noise to node features, ensuring the integrity of the hypergraph structure. Furthermore, the contrastive learning method introduced in this study effectively utilizes the homophily characteristic of hypergraphs, providing a more effective approach than existing HGCL methods. Also, the proposed HyFi model stands out for its efficiency in processing speed and memory usage, making it an ideal solution for various applications.

VII. Acknowledgement

This work was supported by the Technology Innovation Program funded By the Ministry of Trade, Industry & Energy (MOTIE) (No.20022899)

References

[1] S. Bai, F. Zhang, and P. H. Torr, "Hypergraph convolution and hypergraph attention," *Pattern Recognition*, vol. 110, p. 107637, 2021.

[2] D.-Y. Chen, X.-P. Tian, Y.-T. Shen, and M. Ouhyoung, "On visual similarity based 3d model retrieval," in *Computer graphics forum*, vol. 22, no. 3. Wiley Online Library, 2003, pp. 223–232.

[3] E. Chien, C. Pan, J. Peng, and O. Milenkovic, "You are allset: A multiset function framework for hypergraph neural networks," 2021.

[4] D.-A. Clevert, T. Unterthiner, and S. Hochreiter, "Fast and accurate deep network learning by exponential linear units (elus)," 2015.

[5] Y. Dong, W. Sawin, and Y. Bengio, "Hnhn: Hypergraph networks with hyperedge neurons," 2020.

[6] D. Dua, C. Graff *et al.*, "Uci machine learning repository," 2017.

[7] Y. Feng, H. You, Z. Zhang, R. Ji, and Y. Gao, "Hypergraph neural networks," vol. 33, no. 01, pp. 3558–3565, 2019.

[8] X. Gu, L. Chen, and M. Krenn, "Quantum experiments and hypergraphs: Multiphoton sources for quantum interference, quantum computation, and quantum entanglement," *Physical Review A*, vol. 101, no. 3, p. 033816, 2020.

[9] K. Hassani and A. H. Khasahmadi, "Contrastive multi-view representation learning on graphs," pp. 4116–4126, 2020.

[10] X. He, T. Wei, and J. He, "Robust basket recommendation via noise-tolerated graph contrastive learning," in *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*, 2023, pp. 709–719.

[11] J. Huang and J. Yang, "Unignn: a unified framework for graph and hypergraph neural networks," 2021.

[12] D. Lee and K. Shin, "I'm me, we're us, and i'm us: Tri-directional contrastive learning on hypergraphs," vol. 37, no. 7, pp. 8456–8464, 2023.

[13] L. Li, S. Luo, Y. Zhao, C. Shan, Z. Wang, and L. Qin, "Coclep: Contrastive learning-based semi-supervised community search," *IEEE 39th ICDE*, 2023.

[14] Y. Li, H. Chen, X. Sun, Z. Sun, L. Li, L. Cui, P. S. Yu, and G. Xu, "Hyperbolic hypergraphs for sequential recommendation," in *Proceedings of the 30th ACM international conference on information & knowledge management*, 2021, pp. 988–997.

[15] M. Liu, Y. Luo, K. Uchino, K. Maruhashi, and S. Ji, "Generating 3d molecules for target protein binding," *arXiv preprint arXiv:2204.09410*, 2022.

[16] Y. Liu, H. Xuan, B. Li, M. Wang, T. Chen, and H. Yin, "Self-supervised dynamic hypergraph recommendation based on hyper-relational knowledge graph," in *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*, 2023, pp. 1617–1626.

[17] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *International Conference on Learning Representations*, 2019. [Online]. Available: https://openreview.net/forum?id=Bkg6RiCqY7

[18] X. Luo, W. Ju, M. Qu, C. Chen, M. Deng, X.-S. Hua, and M. Zhang, "Dualgraph: Improving semi-supervised graph classification via dual contrastive learning," pp. 699–712, 2022.

[19] Z. Peng, W. Huang, M. Luo, Q. Zheng, Y. Rong, T. Xu, and J. Huang, "Graph representation learning via graphical mutual information maximization," pp. 259–270, 2020.

[20] R. Rossi and N. Ahmed, "The network data repository with interactive graph analytics and visualization," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 29, no. 1, 2015.

[21] K. M. Saifuddin, B. Bumgardner, F. Tanvir, and E. Akbas, "Hygnn: Drug-drug interaction prediction via hypergraph neural network," pp. 1503–1516, 2023.

[22] K. M. Saifuddin, C. May, F. Tanvir, M. I. K. Islam, and E. Akbas, "Seq-hygan: Sequence classification via hypergraph attention network," in *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*, 2023, pp. 2167–2177.

[23] P. Sen, G. Namata, M. Bilgic, L. Getoor, B. Galligher, and T. Eliassi-Rad, "Collective classification in network data," *AI magazine*, vol. 29, no. 3, pp. 93–93, 2008.

[24] F.-Y. Sun, J. Hoffmann, V. Verma, and J. Tang, "Infograph: Unsupervised and semi-supervised graph-level representation learning via mutual information maximization," *arXiv preprint arXiv:1908.01000*, 2019.

[25] L. Sun, Y. Rao, X. Zhang, Y. Lan, and S. Yu, "Ms-hgat: memory-enhanced sequential hypergraph attention network for information diffusion prediction," vol. 36, no. 4, pp. 4156–4164, 2022.

[26] X. Sun, H. Cheng, B. Liu, J. Li, H. Chen, G. Xu, and H. Yin, "Self-supervised hypergraph representation learning for sociological analysis," *IEEE Transactions on Knowledge and Data Engineering*, 2023.

[27] S. Thakoor, C. Tallec, M. G. Azar, M. Azabou, E. L. Dyer, R. Munos, P. Veličković, and M. Valko, "Large-scale representation learning on graphs via bootstrapping," 2022.

[28] P. Veličković, W. Fedus, W. L. Hamilton, P. Liò, Y. Bengio, and R. D. Hjelm, "Deep graph infomax," 2018.

[29] M. Vijaikumar, D. Hada, and S. Shevade, "Hypertenet: Hypergraph and transformer-based neural network for personalized list continuation," pp. 1210–1215, 2021.

[30] Z. Wu, S. Song, A. Khosla, F. Yu, L. Zhang, X. Tang, and J. Xiao, "3d shapenets: A deep representation for volumetric shapes," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2015, pp. 1912–1920.

[31] X. Xia, H. Yin, J. Yu, Q. Wang, L. Cui, and X. Zhang, "Self-supervised hypergraph convolutional networks for session-based recommendation," vol. 35, no. 5, pp. 4503–4511, 2021.

[32] X. Xie, F. Sun, Z. Liu, S. Wu, J. Gao, J. Zhang, B. Ding, and B. Cui, "Contrastive learning for sequential recommendation," pp. 1259–1273, 2022.

[33] N. Yadati, M. Nimishakavi, P. Yadav, V. Nitin, A. Louis, and P. Talukdar, "Hypergcn: A new method for training graph convolutional networks on hypergraphs," *Advances in neural information processing systems*, vol. 32, 2019.

[34] C. Yang, R. Wang, S. Yao, and T. Abdelzaher, "Semi-supervised hypergraph node classification on hypergraph line expansion," in *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*, 2022, pp. 2352–2361.

[35] D. Yang, B. Qu, J. Yang, and P. Cudre-Mauroux, "Revisiting user mobility and social relationships in lbsns: a hypergraph embedding approach," pp. 2147–2157, 2019.

[36] M. Yang, Z. Liu, L. Yang, X. Liu, C. Wang, H. Peng, and P. S. Yu, "Group identification via transitional hypergraph convolution with cross-view self-supervised learning," in *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*, 2023, pp. 2969–2979.

[37] N. Yin, F. Feng, Z. Luo, X. Zhang, W. Wang, X. Luo, C. Chen, and X.-S. Hua, "Dynamic hypergraph convolutional network," pp. 1621–1634, 2022.

[38] Y. You, T. Chen, Y. Sui, T. Chen, Z. Wang, and Y. Shen, "Graph contrastive learning with augmentations," *Advances in neural information processing systems*, vol. 33, pp. 5812–5823, 2020.

[39] Y. You and Y. Shen, "Cross-modality and self-supervised protein embedding for compound–protein affinity and contact prediction," *Bioinformatics*, vol. 38, no. Supplement_2, pp. ii68–ii74, 2022.

[40] J. Zhang, M. Gao, J. Yu, L. Guo, J. Li, and H. Yin, "Double-scale self-supervised hypergraph learning for group recommendation," pp. 2557–2567, 2021.

[41] R. Zhang, T. Zhou, and J. Ma, "Multiscale and integrative single-cell hi-c analysis with higashi," *Nature biotechnology*, vol. 40, no. 2, pp. 254–261, 2022.

[42] H. Zhao, X. Yang, Z. Wang, E. Yang, and C. Deng, "Graph debiased contrastive learning with joint representation clustering." pp. 3434–3440, 2021.

[43] Y. Zhu, Y. Xu, F. Yu, Q. Liu, S. Wu, and L. Wang, "Deep graph contrastive representation learning," 2020.

[44] ——, "Graph contrastive learning with adaptive augmentation," in *Proceedings of the Web Conference 2021*, 2021, pp. 2069–2080.

[45] Z. Zhu, C. Gao, X. Chen, N. Li, D. Jin, and Y. Li, "Inhomogeneous social recommendation with hypergraph convolutional networks," 2022.